

Distributed Query Processing — Exam Notes

Short, memorable summary with simple English + বাংলা concept explanation | Source: Md. Golam Moazzam, Dept. of CSE, JU

1. Basic Definitions

- Query:** A statement that asks the database to retrieve (fetch) some information.
বাংলা: Query মানে হলো ডাটাবেজ থেকে তথ্য চাওয়ার একটি অনুরোধ। যেই ভাষায় এই অনুরোধ লেখা হয় তাকে বলে **Query Language** (যেমন SQL)।
- Query Processor:** The DBMS module that takes a high-level (SQL-like) query and internally converts/optimizes it so the user doesn't need to worry about low-level data storage details.
বাংলা: ইউজার সহজ ভাষায় (SQL) query লেখে, আসল কাজ — কীভাবে data fetch হবে, কোন strategy সবচেয়ে efficient — তা Query Processor নিজেই ঠিক করে। এতে ইউজারকে জটিল optimization নিয়ে ভাবতে হয় না।

2. Main Function / Objective of Query Processor

- Transform a **high-level (Calculus) query** → **low-level (Algebraic) query**, such that both give the **same result** (correct) and use minimum resources (efficient).
বাংলা: User-এর লেখা calculus query (SQL) কে relational algebra query-তে রূপান্তর করাই মূল কাজ। রূপান্তর সঠিক (same result) ও efficient (কম সময়/জায়গা) হতে হবে।

3. Why Query Processing is Hard in Distributed DB

- Data is spread across many sites → processor may need to access multiple sites.
 - Many factors affect performance → response time can become very high.
 - So the problem is broken into smaller sub-problems (steps) to solve easily.
- বাংলা: Centralized DB-তে সব data এক জায়গায়, তাই সহজ। কিন্তু Distributed DB-তে data অনেক site-এ ছড়ানো থাকে, network দিয়ে data আনতে হয় — তাই কষ্টসাধ্য ও সময়সাপেক্ষ। এজন্য পুরো সমস্যাকে ছোট ছোট ধাপে ভাগ করা হয়।

4. Main Problem: Query Optimization

- Query optimization** = choosing the execution strategy that uses the **least cost** (time & resources).
- Cost** depends on: processing time at each site, resource utilization, data transfer time between sites.
- Time & space** both matter — low-cost strategy may need more space, so balance is needed.

5. Important Characteristics of Query Processor

Characteristic	Simple Meaning (+ বাংলা)
Languages	Input = Calculus query (high-level), Output = Algebraic query (low-level, executable). Input SQL -এর মত, output executable algebra ।
Types of Optimization	Compare all strategies by cost; pick the one with least time & space. সব strategy -র cost হিসাব করে সবচেয়ে কম cost -এরটা বেছে নেয়া।
Optimization Timing	Less time to decide optimization = better. Optimization করতে কম সময় লাগলেই ভালো।
Statistics	Uses stats like: no. of fragments, tuple count, distinct values, histograms → helps decide best plan. ডাটাবেজের পরিসংখ্যান (কত fragment , কত row) জানলে ভালো plan বানানো যায়।
Decision Sites	Centralized (1 site decides) vs Distributed (many sites decide together). Strategy কে ঠিক করবে — এক site নাকি একাধিক site মিলে।
Exploitation of Network Topology	Network shape/speed/cost affects performance. নেটওয়ার্কের গঠন ও গতি performance -কে প্রভাবিত করে।
Exploitation of Replicated Fragments	Relations split into fragments; fragments may be copied (replicated) at multiple sites for reliability & faster reads. Relation ভেঙে fragment হয়, আর reliability/read-performance বাড়াতে fragment -এর কপি (replica) রাখা হয় বিভিন্ন site -এ। এই process -কে localization বলে।
Use of Semi-join	Semi-join (⋈) used instead of full join to reduce data sent over network. পুরো join না করে semi-join করলে নেটওয়ার্কে কম data পাঠাতে হয়, তাই communication cost কমে।

6. Main Layers of Distributed Query Processing (VERY IMPORTANT — 4 Layers)

1. Query Decomposition

2. Data Localization

3. Global Query Optimization

4. Distributed Execution

Flow: **Calculus query (global relations)** → Decomposition → **Algebraic query (global relations)** → Localization → **Algebraic query (fragments)** → Global Optimization → **Distributed Query Execution Plan** → Execution → **Result**
→ First 3 layers = done at **Control Site** (uses Global Schema, Fragment Schema, Allocation Schema).
→ Last layer = done at **Local Sites**.

বাংলা: প্রথম ৩টা **layer** একটা **central control site**-এ হয় আর **schema** তথ্য (**global/fragment/allocation schema**) ব্যবহার করে **query rewrite** ও **optimize** করে। শেষ **layer (Execution)** স্থানীয় (**local**) সাইটগুলোতে হয়, যেখানে আসল **fragment** থাকে।

Layer 1: Query Decomposition (4 sub-steps) — মুখস্থ করুন: N-A-E-R

1. Normalization Make query into a standard logical form (CNF/DNF)	2. Analysis Reject type-incorrect / semantically incorrect queries	3. Elimination of Redundancy Simplify by removing redundant predicates	4. Rewriting Convert to relational algebra (Operator Tree)
--	--	--	--

- **Normalization:** Query qualification (WHERE clause) rewritten as **Conjunctive Normal Form (CNF)** = ANDs of ORs, OR **Disjunctive Normal Form (DNF)** = ORs of ANDs.
- **Analysis:** Query rejected if — **type incorrect** (undefined attribute/relation, wrong operation on a type) OR **semantically incorrect** (its **Query Graph** is disconnected → useless join).
 - **Query Graph:** node = relation, edge = join(between 2 relations) or project(to result). **Join Graph** = subgraph showing only join edges.
- **Elimination of Redundancy:** Uses Boolean simplification rules ($P \wedge P = P$, $P \vee \neg P = \text{true}$, $P \wedge \text{false} = \text{false}$, etc.) to shorten the qualification.
- **Rewriting:** Build an **Operator Tree** — leaf = base relation (from FROM clause), root = Project (from SELECT clause), internal nodes = Select/Join (from WHERE clause).

বাংলা: **Query Decomposition centralized** আর **distributed** উভয় সিস্টেমেই একই রকম, কারণ এখানে **global relation** নিয়ে কাজ হয়, **distribution** বিবেচনা করা হয় না।
ধাপগুলো: প্রথমে **query**-কে **normal form**-এ আনা হয়, তারপর ভুল/অসংগত **query** বাতিল করা হয় (**Analysis**), এরপর অপ্রয়োজনীয় শর্ত বাদ দেয়া হয় (**Redundancy elimination**), শেষে **relational algebra**-তে **rewrite** করে **Operator Tree** বানানো হয়।

Layer 2: Data Localization

Input = algebraic query on **global relations**. Goal = replace each global relation with its actual **fragments** (using Fragment Schema) → produces a **Localized Query** on fragments.

- Determines **which fragments** are involved in the query.
- Naive method: replace each leaf (global relation) in the operator tree with a subtree = its **localization program**.

বাংলা: **Global relation** আসলে বাস্তবে অনেকগুলো ছোট ছোট **fragment** হিসেবে বিভিন্ন **site**-এ রাখা থাকে। এই **layer** সেই **global query**-কে **fragment**-ভিত্তিক **query**-তে রূপান্তর করে — অর্থাৎ কোন **fragment** লাগবে, তা বের করে (**localize** করে)।

Layer 3: Global Query Optimization

Input = fragment algebraic query. Goal = find an execution strategy **close to optimal** using Allocation Schema.

- (i) Best **ordering of operations** in the query.
- (ii) Best **communication operations** to minimize a **cost function** (disk space, disk I/O, buffer, CPU cost, communication cost).
- Achieved mainly through **Semi-join** operator (reduces data sent over network) instead of full Join.

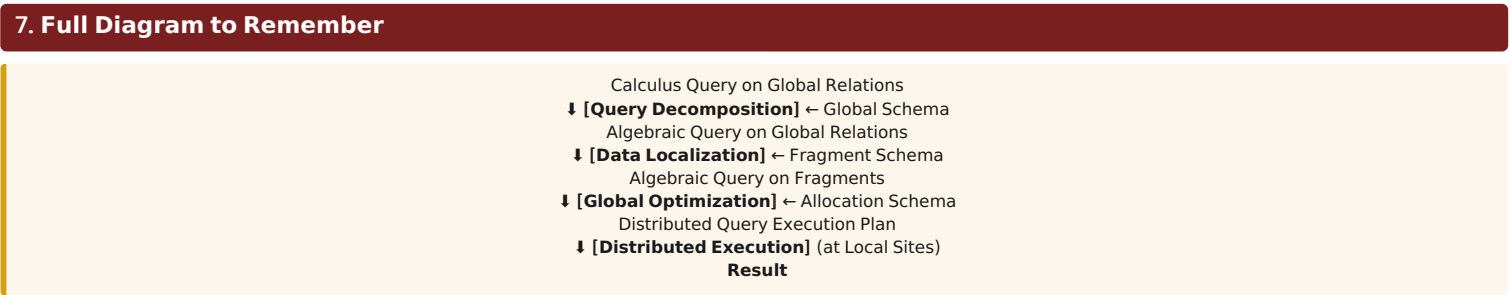
বাংলা: এই ধাপে সবচেয়ে কম **cost**-এর **execution plan** খোঁজা হয় — কোন **operation** আগে করলে ভালো হবে, আর কীভাবে সাইটগুলোর মধ্যে **data** আদান-প্রদান (**communication**) কমানো যায়। এখানে **Semi-join** ব্যবহার করলে পুরো **relation** না পাঠিয়ে শুধু দরকারি অংশ পাঠানো যায়, ফলে **network cost** কমে।

Layer 4: Distributed Execution

Final output = **Distributed Query Execution Plan** executed by **Local Sites** (not the control site alone).

- Each piece of the plan = a **local query**, executed at one site.
- Each local query is further optimized using that site's **local schema** (local DBMS optimizer).

বাংলা: শেষ ধাপে প্রতিটি **sub-query (local query)** নিজ নিজ **site**-এ **execute** হয় এবং সেই **site**-এর **local schema** অনুযায়ী আরেকবার **optimize** করা হয়। সবশেষে ফলাফল একত্র করে **user**-কে দেয়া হয়।



8. Worked Example (Common Exam Question)

Query: "Find names of employees who worked on project P1 for 12 or 24 months"

SQL	SELECT ENAME FROM EMP, ASG WHERE EMP.ENO=ASG.ENO AND ASG.PNO="P1" AND DUR=12 OR DUR=24
CNF	EMP.ENO=ASG.ENO ∧ ASG.PNO="P1" ∧ (DUR=12 ∨ DUR=24)
DNF	(EMP.ENO=ASG.ENO ∧ ASG.PNO="P1" ∧ DUR=12) ∨ (EMP.ENO=ASG.ENO ∧ ASG.PNO="P1" ∧ DUR=24)

বাংলা: এই উদাহরণে দেখানো হয়েছে কীভাবে একই **WHERE condition**-কে **CNF (AND**-এর মধ্যে **OR**) এবং **DNF (OR**-এর মধ্যে **AND**) আকারে লেখা যায় — এটা **Normalization** ধাপের অংশ।

9. Quick Revision — One Line Each (Fill-in-the-blank ready)

- Query Processor's main function = convert calculus query → algebraic query (correct + efficient).
- Main problem in query processing = **Query Optimization**.

- 4 layers = Query Decomposition, Data Localization, Global Query Optimization, Distributed Execution.
- 4 steps of Query Decomposition = Normalization, Analysis, Elimination of Redundancy, Rewriting.
- Query Decomposition + Data Localization together = **Query Rewriting**.
- First 3 layers done at = **Control Site**; last layer done at = **Local Sites**.
- Semi-join used instead of Join to = minimize data communication.
- Query is semantically incorrect if = its Query Graph is **disconnected**.
- Operator Tree: leaf = relation, root = project, internal node = select/join.
- Localization = mapping global relations → physical fragments.

10. Possible Exam Questions (self-test)

1. What is a query and query processor? Explain main function of query processor.
2. What are the main problems of query processing?
3. Describe the important characteristics of a query processor.
4. Draw and explain the generic layering scheme of distributed query processing.
5. Explain the four steps of query decomposition with example.
6. What is a query graph / join graph? When is a query semantically incorrect? Give example.
7. What is data localization? Why is it needed?
8. What does global query optimization achieve? Role of semi-join.
9. Explain distributed execution — who performs it and how?
10. Convert a given SQL query to CNF/DNF and draw its operator tree.